Ashwath Raj M | AP25110010712 | Core CSE A

GITHUB LINK: https://github.com/Ashwath-Raj/CSE-204-DAA-assignments-

1. All-Pair Shortest Path Using Floyd-Warshall Algorithm

Problem

Implement the Floyd-Warshall algorithm to find the shortest path distance between every pair of vertices in a weighted graph.

Method

The algorithm initializes a distance matrix from the adjacency matrix. It then considers each vertex k as an intermediate vertex and updates dist[i][j] when the path from i to j through k is shorter. A value of -1 in the input denotes no edge; diagonal distances are set to 0.

Output:

```
$ ./floyd_warshall
Enter Number of Vertices: Enter Adjacency Matrix (-1 for no edge):
Shortest Distance Matrix:
     0     5     8     9
   INF     0     3     4
   INF   INF     0     1
   INF   INF   INF     0
```

Sample input

```
4
0  5  -1  10
-1  0   3  -1
-1 -1   0   1
-1 -1  -1   0
```

Code:

```
/*
Program Name: All-Pair Shortest Path Using Floyd-Warshall Algorithm
Program Description: Finds shortest distances between every pair of vertices in a weighted directed graph.
Program Creation Time: Oct 24th, 2026
Program Creator: Ashwath Raj | AP25110010712
*/
#include<iostream>
#include<vector>
#include<iomanip>
using namespace std;

const int INF=1000000000;

int main(void){
    int V;
    cout<<"Enter Number of Vertices: ";
    cin>>V;

    vector<vector<int>> dist(V,vector<int>(V));
    cout<<"Enter Adjacency Matrix (-1 for no edge):"<<endl;
    for(int i=0;i<V;i++){
        for(int j=0;j<V;j++){
            int x;
            cin>>x;
            dist[i][j]=(i==j)?0:(x==-1?INF:x);
        }
    }

    for(int k=0;k<V;k++)
        for(int i=0;i<V;i++)
            for(int j=0;j<V;j++)
                if(dist[i][k]!=INF && dist[k][j]!=INF &&
                   dist[i][k]+dist[k][j]<dist[i][j])
                    dist[i][j]=dist[i][k]+dist[k][j];

    cout<<"Shortest Distance Matrix:"<<endl;
    for(int i=0;i<V;i++){
        for(int j=0;j<V;j++){
            if(dist[i][j]==INF) cout<<setw(6)<<"INF";
            else cout<<setw(6)<<dist[i][j];
        }
        cout<<endl;
    }
    return 0;
}
```

Complexity

Time Complexity: $O(V^3)$. Space Complexity: $O(V^2)$. The three nested loops examine all intermediate, source, and destination vertices. The distance matrix requires $O(V^2)$ space.